
SHIFT: Sigmoid-Based Heuristic Invertible Fitness-Landscape Transformation for Accelerating SBST*

Jeongjin Han¹ Seunghoon Sim¹ Jian Lee¹ Seongyoon Park¹

¹School of Computing, KAIST
hjj22@kaist.ac.kr

Abstract

Search-Based Software Testing (SBST) automates test input generation but is frequently hindered by challenging fitness landscapes characterized by numerous deceptive local optima that impede search progress, as well as extended plateaus where informative fitness signals are scarce. To address this bottleneck, we propose SHIFT (Sigmoid-Based Heuristic Invertible Fitness-Landscape Transformation for Accelerating SBST), a method designed to compress local landscapes and facilitate escape from stagnant regions without altering global semantics. By systematically contracting dense regions where search points cluster, the approach preserves mapping invertibility while enabling optimization algorithms to traverse more effectively toward global coverage with the same step size. When evaluated against established baselines—pure hill climbing and genetic algorithms—under a normalized experimental protocol, the proposed technique yields substantial improvements in convergence speed and search efficiency. These results demonstrate that sigmoid compression constitutes a lightweight yet effective mechanism for achieving faster and more reliable coverage discovery in complex testing environments.

Keywords: Search-Based Software Testing (SBST), Fitness Landscape Analysis, Sigmoid Compression, Local Optima, Automated Test Generation

1 Introduction

Software testing remains one of the most resource-intensive tasks in software engineering, particularly when manual test design must account for complex input spaces and diverse program behaviors. Search-Based Software Testing (SBST) aims to alleviate this burden by framing test generation as an optimization problem, employing metaheuristics to search for inputs that trigger new coverage automatically. Among these techniques, Hill Climbing (HC) is widely used due to its simplicity, low computational overhead, and strong performance on smooth fitness landscapes.

However, the efficacy of hill climbing degrades significantly when the underlying search space is rugged or full of plateaus. Real-world software often produces fitness landscapes characterized by cliffs, deceptive plateaus, and clusters of local optima, especially around branch conditions involving complex logic, multi-variable interactions, or discontinuous boundary checks. Once the search falls into one of these regions, local fitness updates vanish, and the algorithm repeatedly revisits neighbors that yield no meaningful progress. In practice, this results in prolonged stagnation phases, wasted evaluation budgets, and an overall failure to reach deeper, hard-to-cover branches.

These limitations expose a structural mismatch: hill climbing is inherently myopic and performs best on landscapes where unproductive local regions are sparse and global transitions are accessible. Rather than attempting to render the hill climbing algorithm more sophisticated through complex

*Code and data available at: <https://github.com/Jeong-jin-Han/SHIFT-SBST.git>

operator variations or surrogate fitness estimators, we instead adopt a more fundamental approach: directly reshaping the geometry of the search space itself. This paper introduces *SHIFT* (Sigmoid-Based Heuristic Invertible Fitness-Landscape Transformation for Accelerating SBST), a framework that compresses the fitness landscape to address this mismatch. By modifying the landscape geometry, we enable standard hill climbing to operate more effectively using its existing mechanics, without altering the global semantics of the test input domain.

Compressing local regions using a smooth, invertible sigmoid transformation serves to narrow flat basins, reduce the dominance of small-scale local optima, and "stretch" transitions toward more promising areas. Consequently, the search is less likely to become trapped early and obtains a clearer gradient toward global coverage-relevant zones. Unlike discrete mutation adjustments, this transformation uniformly reshapes the region around the current search point, enabling more meaningful exploratory steps without significantly increasing computational overhead.

The remainder of this paper presents the motivation for the *SHIFT* model, describes the mathematical design of the sigmoid-based compression that forms the core of our approach, and evaluates its impact on coverage efficiency by comparing its performance against standard hill climbing and a genetic algorithm as baselines. The experimental results demonstrate that compressing the solution space constitutes a lightweight, practical, and effective strategy for recovering hill climbing’s effectiveness in rugged SBST scenarios.

2 Related Works

Research in Search-Based Software Testing (SBST) has explored various strategies to mitigate the difficulties posed by rugged or deceptive fitness landscapes. Although prior work does not provide an invertible transformation directly comparable to our SHIFT, several studies attempt to reshape or reinterpret the landscape in ways that improve search effectiveness.

A notable example is the *surF* methodology proposed by Manzoni et al. [2020], which applies a discrete Fourier transform and filters out high-frequency components, such as sharp spikes, to smooth the fitness landscape. By suppressing these high-frequency signals, *surF* constructs a smoother surrogate landscape while preserving the location of the global optimum. This approach demonstrates that smoothing or reshaping the evaluation function can make local search algorithms more effective, even though the surrogate itself is not strictly invertible back to the original domain. Their work highlights that modifying the landscape can reduce the number of local optima and help search algorithms escape deceptive regions, supporting the conceptual motivation behind our compressive mapping.

Comparative studies between search heuristics in SBST also shed light on why hill climbing benefits from such landscape shaping. Earlier foundational work by Harman and McMinn [2010] compared hill climbing with multi-population GAs for structural test generation. They identified problem classes where GAs outperform HC, particularly when the fitness structure resembles a “Royal Road” landscape with decomposable building blocks, where the crossover operator can significantly benefit. However, they also showed that HC can be competitive or even superior when the landscape is fragmented or discontinuous. This further motivates the need to reshape the landscape for HC rather than rely solely on algorithmic complexity.

Tabu Search has also been applied to test-case generation, primarily as a mechanism to avoid cycling and escape shallow local optima. Ma et al. [2022] demonstrate a tabu-based path exploration strategy that maintains a memory of recently visited test inputs to prevent re-evaluating unproductive regions. While their approach does not perform dimension-wise tabu operations—which aligns more closely with our compression model—it illustrates that memory-based mechanisms can help push search beyond local traps. However, the tabu list in existing SBST work is applied to complete test inputs rather than individual parameter dimensions, and the literature presents no known instance of tabu applied at the per-axis granularity. This absence highlights a gap that our SHIFT model partially addresses: instead of forbidding individual points or parameter values, our method geometrically reshapes the search space to make problematic regions less dominant and reduce the need for explicit memory.

Overall, prior work converges on a common theme: when the fitness landscape is rugged or poorly structured, algorithmic improvements alone seldom yield substantial gains. Whether through Fourier smoothing, empirical comparisons of HC and GA, or tabu-based memory mechanisms, the literature consistently points to landscape structure as the core limiting factor. Our work builds on this insight by providing a direct, invertible transformation that compresses local regions and exposes broader global transitions, enabling simple hill climbing to recover its effectiveness without reliance on heavy evolutionary machinery or complex hyperparameter tuning.

3 Methodology

3.1 Terminology and the Proposed SHIFT Framework

For clarity, we introduce the terminology used throughout this paper when contrasting our approach with baseline search algorithms. At the core of our method lies **SHIFT**, a sigmoid-based, invertible transformation framework designed to reshape the fitness landscape encountered in SBST. SHIFT applies axis-aligned warpings that compress locally flat or weakly varying basins—regions in which classical local search becomes trapped—while preserving a one-to-one correspondence between the original and transformed search spaces. Because these transformations remain lightweight and invertible, the framework can be integrated into existing metaheuristics without altering their algorithmic structure.

When combined with standard hill climbing, SHIFT yields the proposed method **HC-SHIFT**. The underlying hill-climbing procedure is preserved, but the added transformation enables the search to escape local minima in a controlled manner by selectively compressing basins along dimensions identified as relevant to fitness improvement.

For comparison, we refer to two baselines: **HC**, denoting the unmodified hill-climbing algorithm, and **GA**, denoting the genetic algorithm without any landscape transformation. Throughout the remainder of this report, these terms are used consistently across tables, figures, and experimental discussions; unless stated otherwise, HC and GA indicate the non-transformed baselines, while HC-SHIFT refers to our proposed approach.

3.2 Program Instrumentation and Fitness Evaluation

Our framework follows the standard Search-Based Software Testing (SBST) pipeline: (1) instrument the program under test (PUT), (2) collect branch execution information, and (3) compute fitness for each input candidate. Instrumentation is performed via a Python source-to-source transformation that injects branch probes directly into the Abstract Syntax Tree (AST) of the PUT. All instrumentation and fitness logic is implemented in `sbst_core.py`.

AST-based Instrumentation.

Given a Python function, we parse its source into an AST and apply a transformer that rewrites each Boolean expression into a call to a branch-probe runtime (`BranchProbe`). Each conditional construct—including `if`, `while`, `for`, pattern matching, chained comparisons, and logical connectives—is assigned a unique branch identifier (`bid`). During execution, the probe records both the Boolean outcome and the tightest observed branch-distance values for the True and False directions, producing a complete mapping

$$\text{bid} \mapsto \{\text{outcome}, d_{\text{true}}, d_{\text{false}}\}$$

for each evaluated input. Additionally, for every branch we compute a *guard chain*, i.e., the list of ancestor branch conditions that must be satisfied before the target branch can be reached. These guard chains, collected during AST traversal, are later used to compute the approach level.

Branch-Distance Computation.

For elementary comparisons such as $a < b$ or $x == y$, we follow the standard SBST branch-distance formulation. Given operands a, b and operator op , we define

$$\text{raw_f}(a, b, op) = \begin{cases} (a - b) + 1, & a < b, \\ (b - a) + 1, & a > b, \\ |a - b|, & a = b, \\ -|a - b|, & a \neq b, \end{cases}$$

which is then mapped to a non-negative branch distance $d \in [0, \infty)$ using operator-specific rules (with $d = 0$ indicating that the condition is satisfied). These rules are implemented in `raw_f`, `bd`, and `b_from_raw`. The framework further extends branch-distance computation to element-wise comparison of tuples or lists, Boolean operators (`and`, `or`, `not`), membership expressions via interval compression (`in/not in`), and pattern-matching constructs by conditional lifting.

Approach Level (AL).

Let $\mathcal{G} = [(g_1, v_1), \dots, (g_k, v_k)]$ denote the ordered list of ancestor guards required to reach branch bid, where each v_i is either True or False. If i^* is the index of the first unsatisfied guard, then the approach level is defined as

$$AL(\text{bid}) = |\mathcal{G}| - i^*,$$

with $AL = 0$ only if all guards are satisfied.

Normalised Branch Distance (nBD).

Once the failing guard (or the target guard itself) is identified, its raw distance d is normalized using

$$\text{nBD}(d) = 1 - 1.001^{-d},$$

which yields a smooth value in $[0, 1)$ that decreases monotonically as the input approaches the required branch direction.

Final Fitness Function.

The overall fitness for branch b and desired direction $\text{want_true} \in \{0, 1\}$ is computed as

$$F = AL + \text{nBD},$$

with $F = 0$ indicating that the branch was executed in the desired direction. This behavior is implemented in `fitness_AL()`, which manages guard handling, approach-level computation, and branch-distance aggregation.

Execution and Trace Collection.

The instrumented PUT is executed in an isolated probe environment. Before each execution the probe resets its internal state, and during execution it records branch outcomes and branch-distance values at every conditional evaluation. The collected trace is then passed directly into the fitness computation without the need for additional runtime instrumentation.

Handling Unreachable Loop Branches.

Certain branch directions are semantically unreachable under Python’s loop semantics and are therefore excluded during target generation. For `while True` loops, the False direction—representing loop exit—cannot occur unless an explicit `break` is present. The AST transformer detects constant-True loop conditions (line 1140 of `sbst_core.py`) and marks such branches accordingly (lines 1152–1153). During target enumeration, `make_targets_for_func()` (lines 1965–1968) automatically omits the unreachable False direction.

Similarly, for `for`-loops the False branch corresponds to the loop body not executing even once. Using iteration metadata tracked through `loop_minlen` (line 544), the framework determines cases where the iterable is guaranteed to be nonempty; `make_targets_for_func()` (lines 1970–1978) omits such False-direction targets as they are unsatisfiable.

By filtering out unreachable loop branches, the framework avoids expending search effort on infeasible targets and ensures that fitness evaluation focuses solely on semantically meaningful branches.

3.3 Search Algorithms

3.3.1 Baseline: Simple Hill Climbing (HC)

As a baseline local search method, we employ a deterministic, coordinate-wise hill climbing algorithm, hereafter referred to as **HC**. This baseline operates directly on the original fitness landscape introduced in Section 3.2, where each candidate input is represented as an n -dimensional integer vector and evaluated using the fitness function $F = AL + nBD$. The full implementation is provided in `hill_climb_simple_nd_code` (lines 827–902 of `hill_climb_multiD.py`).

Overview. Simple hill climbing (HC) serves as the baseline local-search method in our evaluation. The algorithm performs deterministic coordinate-wise descent on the original fitness landscape, evaluating the $2n$ axis-aligned neighbors obtained by adding or subtracting one unit in a single dimension. At each step, HC selects the neighbor with the lowest fitness and moves only when an improvement is found; otherwise, it terminates at a local minimum. The procedure operates under the same per-branch time budget of $T = 20$ seconds used for GA and HC-SHIFT, and also obeys a maximum step limit of $K = 2000$. Because HC explores neither diagonal directions nor alternative basins, and lacks any mechanism for escaping plateaus or rugged regions where multiple neighbors share identical fitness, it serves as a minimal but informative baseline against which the behavior of GA and HC-SHIFT can be contrasted.

(a) Neighbor Generation and Search Strategy.

At each iteration, HC considers exactly $2n$ axis-aligned neighbors obtained by applying a fixed ± 1 perturbation to a single coordinate:

$$\mathcal{N} = \{(x_1, \dots, x_d - 1, \dots, x_n), (x_1, \dots, x_d + 1, \dots, x_n)\}_{d=1}^n.$$

No diagonal moves, adaptive step sizes, or multi-coordinate perturbations are used; the search progresses solely through these minimal coordinate-wise increments (lines 868–888). Because all $2n$ neighbors may share identical fitness values on plateaus, HC has no mechanism to escape such regions and is therefore highly sensitive to flat or rugged landscapes.

Among all evaluated neighbors, HC selects the candidate with the lowest fitness (steepest descent, lines 891–894):

$$x' = \arg \min_{y \in \mathcal{N}} F(y).$$

If $F(x') < F(x)$, the search moves to x' and continues; otherwise it terminates at a local minimum (lines 896–900).

(b) Time-Budget Enforcement.

Like GA (Section 3.3.2), our HC implementation enforces a strict per-branch time budget to ensure fair comparison across all methods. Before evaluating each neighbor—and before the initial evaluation—HC checks whether the elapsed time has exceeded the budget (lines 853–856, 862–865, 870–873, 880–883):

$$\text{if } (t_{\text{current}} - t_{\text{start}}) \geq t_{\text{max}} \Rightarrow \text{terminate and return best.}$$

If the time limit is reached at any point, HC immediately returns the current best solution, even if the step limit has not been exhausted.

(c) Stopping Conditions.

HC terminates under three circumstances. First, if none of the $2n$ neighbors yields a strictly lower fitness value, the algorithm concludes that it has reached a local minimum and halts (line 900). Second, the search stops when the maximum step limit of $K = 2000$ iterations is reached (line 861). Finally, HC enforces a strict per-branch time budget and immediately terminates once the allotted time has elapsed. In all termination cases, the algorithm returns the best point encountered up to that moment, represented as the trajectory $\{(x_0, F(x_0)), (x_1, F(x_1)), \dots\}$ (lines 859, 898).

3.3.2 Baseline: Genetic Algorithm (GA)

As a second baseline, we employ a standard genetic algorithm, hereafter referred to as **GA**. The GA operates over the same n -dimensional integer input domain and uses the fitness function $F = AL + nBD$ defined in Section 3.2. Our implementation follows a steady-state evolutionary scheme with tournament selection, uniform crossover, independent per-gene mutation, and elitist survivor selection. The full implementation is provided in `ga.py` (lines 20–246). Importantly, no SHIFT-based transformations are applied—GA serves purely as a population-based global-search method for comparison.

Overview. The genetic algorithm (GA) follows a standard steady-state evolutionary procedure that iteratively refines a population of candidate inputs. The algorithm begins by generating an initial population using either random or biased initialization and evaluating all individuals under the same per-branch time budget applied to HC and HC-SHIFT. Each generation then consists of tournament-based parent selection, uniform crossover, and independent per-gene mutation, with at least one mutation enforced to avoid genetic stagnation. Elitist survivor selection preserves the top-performing individuals, while deduplication removes redundant genotypes and helps maintain diversity on flat landscapes. At every stage of evaluation—including initialization, the start of each generation, and during offspring evaluation—the GA enforces early stopping: if any individual achieves fitness $F = 0$, the algorithm terminates immediately. Otherwise, evolution continues until the time budget is exhausted, no further improvements occur, or the generation limit is reached, at which point the best individual encountered is returned.

(a) Representation and Initialization.

Each candidate input is represented as an integer vector $x \in \mathbb{Z}^n$ constrained by the automatically inferred bounds $[L, H]$ of the instrumented program.

The initial population of size $P = 10000$ is generated using either **random** or **biased initialization** (see Section 4.4.2 for details). Our experiments use biased initialization by default, sampling near program constants to accelerate early convergence when control-flow predicates depend on specific literals (e.g., `if x > 100`).

(b) Selection, Crossover, and Mutation.

Selection. We use k -way tournament selection with $k = 3$ to choose parents (lines 103–106). For each parent, k individuals are sampled uniformly at random from the current population, and the one with the lowest fitness is selected.

Crossover. Given two selected parents $p_1, p_2 \in \mathbb{Z}^n$, uniform crossover produces an offspring $c \in \mathbb{Z}^n$ by independently choosing each gene from one parent or the other with equal probability:

$$c_i = \begin{cases} (p_1)_i & \text{with prob. 0.5,} \\ (p_2)_i & \text{with prob. 0.5,} \end{cases}$$

for every dimension $i \in \{1, \dots, n\}$ (lines 108–110).

Mutation. Mutation is applied independently to each gene with probability $p_{\text{mut}} = 1/n$ (lines 112–123). When a gene is mutated, it is perturbed by a randomly chosen integer step from the set $\{-3, -2, -1, +1, +2, +3\}$ and clipped to remain within $[\ell, u]$:

$$x'_i = \text{clip}(x_i + \Delta), \quad \Delta \in \{-3, -2, -1, +1, +2, +3\}.$$

To prevent genetic stagnation on flat fitness regions, we enforce at least one mutation per offspring (`ensure_mutation=True`, lines 120–122): if no gene was mutated during the per-gene pass, we forcibly mutate a randomly selected gene.

(c) Elitism and Diversity Control.

After fitness evaluation, the population is sorted by fitness, and the top $\lceil P \cdot \alpha_e \rceil$ individuals (elite ratio $\alpha_e = 0.1$, i.e., 10% of the population) are preserved verbatim into the next generation (lines 183–185). This elitist strategy guarantees monotonic improvement of the best solution across generations.

To maintain population diversity and reduce redundant fitness evaluations, all duplicate individuals are removed via deduplication before the next evaluation cycle (lines 125–131, 200). This is particularly important on flat landscapes where many candidates may converge to the same point.

(d) Time-Budget Enforcement and Early Stopping.

Unlike fixed-generation genetic algorithms, our implementation enforces a strict per-branch time limit identical to that used by HC (Section 3.3.1) and HC-SHIFT (Section 3.3.3). Before evaluating any candidate—both during initialization and within each generation—the GA checks whether the elapsed time has exceeded the allowed budget (lines 143–154, 206–212):

$$\text{if } (t_{\text{current}} - t_{\text{start}}) \geq t_{\text{max}} \quad \Rightarrow \quad \text{terminate and return best.}$$

This mechanism guarantees that all three methods (HC, GA, HC-SHIFT) operate under identical computational constraints and that their performance can be compared fairly.

In addition to time-budget enforcement, the GA incorporates an early-stopping criterion: once any individual achieves fitness $F = 0$, indicating that the target branch has been exercised in the desired direction, the algorithm terminates immediately. Early stopping is applied consistently during the initial evaluation of the population (lines 167–170), at the beginning of each generation (lines 188–191), and during the evaluation of newly generated offspring (lines 229–233). In all such cases, the GA returns the discovered solution without expending the remaining time or generation budget.

3.3.3 Proposed: HC-SHIFT (N-Dimensional Compression Hill Climbing)

Our main contribution is an n -dimensional hill climbing algorithm that dynamically reshapes the search landscape by detecting and compressing one-dimensional fitness basins. Rather than navigating the original integer-valued space directly, the algorithm constructs an auxiliary nonlinear coordinate system in which large flat regions are contracted into short intervals. This transformation enables efficient escape from plateaus and slow-gradient regions that frequently arise in SBST fitness landscapes.

We refer to this sigmoid-based, invertible transformation framework as **SHIFT** (Sigmoid-Based Heuristic Invertible Fitness-Landscape Transformation for Accelerating SBST), and to the resulting algorithm as **HC-SHIFT**. The complete implementation is available in `compression_hc.py`.

Overview. The $(AL+nBD)$ fitness formulation (Section 3.2) frequently produces extended plateaus and broad basins in which many consecutive inputs share identical fitness. Classical hill climbing (HC, Section 3.3.1) becomes trapped in such regions because all neighboring points appear equally promising. Techniques such as tabu search or simulated annealing escape through randomness or memory, yet they leave the underlying fitness geometry unchanged. HC-SHIFT takes a different approach: it explicitly reshapes the search space by identifying the extent of flat or weakly varying regions and compressing them into a smaller geometric footprint, thereby introducing new gradients that guide search progress.

Algorithm 2 summarizes the overall procedure. Like HC and GA (Section 3.3.2), HC-SHIFT adheres to a strict per-branch time budget of $T = 20$ seconds (Section 4.4.1); before each fitness evaluation, the algorithm checks for timeout and terminates early if necessary to ensure fair comparison. The algorithm begins by marking all dimensions as active and initializing an empty compression manager. It then enters the main hill-climbing loop (Algorithm 5), where it generates axis-aligned and diagonal neighbors in compressed space (Algorithm 6), tracks which dimensions contribute to fitness changes, and deactivates stagnant dimensions after p consecutive steps.

When the hill-climbing loop can no longer make progress, HC-SHIFT performs bidirectional basin detection on each active dimension (Algorithm 3). These probes estimate 1D basin boundaries, update compression metadata, and construct SHIFT warpings w_d that collapse flat regions into shorter intervals in Z -space. Finally, the algorithm evaluates candidates at the detected basin boundaries and restarts from the best point (Algorithm 4). This cycle repeats until convergence ($F = 0$), the set of active dimensions becomes empty, or the time budget expires.

By combining fitness-driven basin detection, invertible space compression, dimension-level pruning, and systematic boundary restarts, HC-SHIFT provides a cohesive landscape-shaping framework that enables efficient navigation of complex n -dimensional search spaces.

(a) 1D Basin Detection.

Given a local minimum x^* along dimension d , HC-SHIFT performs a bidirectional probing procedure to identify a contiguous interval $B_d = [\ell_d, r_d]$ representing the flat or near-flat basin surrounding x^* . Starting from x^* , the algorithm evaluates points one step at a time in both directions and classifies each point according to the following rules:

- **Rule 1 (Equal fitness).** The fitness matches $F(x^*)$; the point is recorded as part of the basin, and probing continues.
- **Rule 2 (Worse fitness).** The fitness is worse but not better than $F(x^*)$; the point is ignored but probing continues in search of a possible exit.
- **Rule 3 (Better fitness).** The fitness strictly improves; probing stops immediately in that direction, and the point is marked as a basin boundary.

During scanning, HC-SHIFT applies a priority policy for determining the basin boundary. If a Rule 3 point is encountered, its location is taken as the boundary for that direction. If no such point appears, the boundary is assigned to the farthest Rule 1 point, representing the maximal extent of the flat region. If neither Rule 1 nor Rule 3 occurs before the probing limit is reached—i.e., only Rule 2 points are observed—the algorithm concludes that no meaningful basin exists on that side.

To prevent unbounded scanning on long plateaus or oscillatory regions, the probing distance is capped by a dimension-specific limit. The full detection logic is implemented in `detect_compression_basin()` (lines 126–224 of `compression_hc.py`) and is invoked by Algorithm 3. If the resulting interval satisfies $|B_d| < 2$, HC-SHIFT skips compression for dimension d because the detected region is too small to constitute a meaningful basin.

(b) Sigmoid Warping (X-space $\rightarrow$ Z-space).

For each detected basin $B_d = [s, s + L - 1]$ of length L , HC-SHIFT builds a smooth warping function $w_d : \mathbb{R} \rightarrow \mathbb{R}$ based on a centered sigmoid:

$$z = w_d(x) = \begin{cases} x & \text{if } x < s, \\ s + \sigma\left(\alpha\left(\frac{x-s}{L} - \frac{1}{2}\right)\right) & \text{if } s \leq x \leq s + L, \\ x - (L - 1) & \text{if } x > s + L, \end{cases}$$

where $\alpha = 5.0$ controls the steepness and $\sigma(t) = 1/(1 + e^{-t})$ is the standard sigmoid function. This mapping contracts the full basin B_d of length L into roughly one unit in z -space while keeping the exterior regions affine (shifted by $L - 1$ on the right to maintain continuity). The inverse function w_d^{-1} uses the logit to map compressed coordinates back into valid integer positions in the original X -space. These transformations are implemented by the `SigmoidWarping` class (lines 15–74) and constitute the core of the SHIFT framework.

(c) Metadata in Original X-Space.

All compression metadata—specifically the basin boundaries (s, L) —is stored directly in the original X -space rather than in the transformed Z -space. Maintaining metadata in X -space ensures that all compression decisions remain aligned with the true fitness function F , prevents inconsistencies that could arise from repeatedly applying warpings, and supports stable behavior across multiple compression cycles. This design also enables conflict-free merging of overlapping basins through the routine `merge_overlapping_compressions` (lines 231–260), which consolidates adjacent or intersecting compressed regions into a coherent representation. The responsibilities associated with managing these structures are handled by `MetadataCompressionOriginalSpace` (lines 76–121) together with `CompressionManagerND` (lines 267–317), which jointly maintain the state of all SHIFT-related metadata across dimensions.

(d) Multi-D Compression Manager and Active Dimensions.

To extend basin compression to the multi-dimensional setting, HC-SHIFT maintains a dynamically updated set of *active dimensions* $\mathcal{A} \subseteq \{0, \dots, n - 1\}$ —those axes along which recent movement, basin detection, or fitness changes suggest the presence of exploitable structure. Initially all dimensions are active, but the set is progressively refined as the search proceeds. During each hill-climb step (Algorithm 5), HC-SHIFT identifies the subset of dimensions $\mathcal{M}$ that contributed to any observed fitness improvements (line 12). For every dimension $d \notin \mathcal{M}$, a stagnation counter σ_d is incremented, whereas counters for dimensions in $\mathcal{M}$ are reset to zero. When σ_d exceeds a patience threshold p (default $p = 20$), dimension d is removed from the active set (line 14), indicating that further exploration along that axis is unlikely to yield progress.

This mechanism ensures that only active dimensions participate in neighbor generation (Algorithm 6), basin detection (Algorithm 3), and restart selection (Algorithm 4). As a result, the effective per-iteration cost is reduced from $O(n)$ to $\tilde{O}(k)$, where $k = |\mathcal{A}| \ll n$ denotes the number of remaining active dimensions. Inactive dimensions retain their previously computed compression metadata but are no longer revisited unless reactivated later in the search.

For each active dimension d , the compression manager maintains a slice-based map

$$\text{dim_compressions}[d] : (\text{fixed coords}) \mapsto \{(s_1, L_1), (s_2, L_2), \dots\},$$

where “fixed coords” represents all coordinates other than d . This design supports heterogeneous and slice-dependent compression geometries, enabling HC-SHIFT to adapt to highly anisotropic and irregular fitness landscapes. Unlike tabu-style or AVM-style strategies that treat all coordinates uniformly or rely solely on historical movement patterns, HC-SHIFT allocates computation selectively, focusing effort precisely on the axes that reveal actionable geometric structure.

(e) Search Procedure and Neighbor Selection.

The HC-SHIFT search procedure alternates between hill climbing in the compressed Z -space, basin detection in the original X -space, and restarts from basin boundaries. During a hill-climb phase (Algorithm 5), neighbor candidates are generated according to Algorithm 6. For each active dimension d , the current coordinate x_d is mapped into compressed form $z_d = w_d(x_d)$ whenever a compression mapping exists. Single-step moves $z_d \pm 1$ are then considered, and the resulting points are mapped back using the inverse transform w_d^{-1} :

$$x'_d \in \{ w_d^{-1}(z_d - 1), w_d^{-1}(z_d + 1) \}.$$

When no compression is associated with dimension d , these updates reduce naturally to $x_d \pm 1$. As a consequence, seemingly small moves in the compressed space may correspond to long jumps along flat basins in the original landscape.

In addition to axis-aligned moves, HC-SHIFT constructs diagonal neighbors by simultaneously perturbing pairs of active dimensions. For any pair (d_1, d_2) , the corresponding compressed coordinates (z_{d_1}, z_{d_2}) are perturbed independently by ± 1 and then mapped back:

$$(x'_{d_1}, x'_{d_2}) = (w_{d_1}^{-1}(z_{d_1} \pm 1), w_{d_2}^{-1}(z_{d_2} \pm 1)).$$

This mechanism produces $O(|\mathcal{A}|^2)$ diagonal candidates and allows the search to traverse narrow or curved passages that would be difficult to escape using only axis-aligned moves.

Whenever hill climbing reaches a point at which none of the generated neighbors reduces the fitness, basin detection is invoked (Algorithm 3) to identify flat regions along each active dimension. For each detected basin $B_d = [\ell_d, r_d]$, restart candidates are placed at the geometric boundaries $\ell_d - 1$ and $r_d + 1$ (Algorithm 4), enabling the algorithm to reposition the search directly outside the compressed plateau rather than stepping through it incrementally.

Among all evaluated candidates, HC-SHIFT adopts the steepest-descent strategy and moves toward the neighbor with the smallest fitness value (lines 8–16 of Algorithm 5). If a strictly better neighbor exists, the algorithm continues its ascent in the compressed space; otherwise, basin detection is triggered. Convergence speed is recorded as the number of hill-climb steps executed in Z -space, a metric that often underestimates the effective distance traveled in the original X -space because SHIFT compresses large plateaus and oscillatory regions into compact intervals.

(f) Comparison with Tabu Search and Related Work.

HC-SHIFT differs fundamentally from tabu search in both purpose and mechanism. Whereas tabu search modifies the search *trajectory* by maintaining a memory structure that prevents revisits, HC-SHIFT instead modifies the *search space* itself. Through its SHIFT transformation, flat regions are compressed into short intervals, enabling the algorithm to traverse large plateaus with only a few effective steps in Z -space. Tabu search reduces cycling but does not alter the underlying geometry of the landscape and does not reduce the effective dimensionality of the problem. In contrast, HC-SHIFT incorporates dimension-level pruning via stagnation tracking, allowing it to focus computation on the axes that meaningfully influence progress.

Relative to prior SBST techniques such as AVM variants, adaptive step-size schemes, and gradient-smoothing methods, HC-SHIFT introduces a distinct landscape-shaping perspective. It performs explicit, fitness-driven probing of flat regions to estimate basin boundaries through bidirectional detection; it applies smooth and invertible sigmoid-based warpings rather than discrete mutational shifts; and it maintains all compression metadata in the original X -space to ensure stability across repeated transformations. Additionally, HC-SHIFT applies per-dimension compression independently while dynamically selecting active dimensions based on stagnation counters, thereby avoiding unnecessary compression on irrelevant axes. The neighbor-generation strategy in compressed space further leverages diagonal movements, enabling escape from narrow transition corridors that commonly trap local-search methods. Finally, HC-SHIFT preserves and accumulates compression information across iterations for the same target branch—such as basin boundaries, active dimensions, and SHIFT parameters—allowing the algorithm to operate on increasingly informed landscapes rather than restarting from scratch at each attempt.

Taken together, these mechanisms constitute a distinct form of landscape-shaping local search that proves highly effective on the rugged and plateau-rich fitness functions frequently encountered in SBST. A detailed empirical comparison with HC and GA is provided in Section 4.

4 Experiment

4.1 Pilot Study on Synthetic Fitness Landscapes

Prior to the main evaluation, we conduct a pilot study on synthetic fitness landscapes to verify the fundamental behavior of SHIFT. This study examines how compression accumulates across trials and how this accumulation affects convergence on landscapes of varying ruggedness and dimensionality. Across all controlled landscapes considered, SHIFT consistently exhibits superior convergence behavior, requiring substantially fewer restarts than HC and surpassing GA in all evaluated settings. SHIFT’s compression mechanism progressively enhances trial effectiveness, enabling reliable escape from rugged or high-dimensional fitness basins.

Synthetic Fitness Functions We evaluate eight synthetic fitness landscapes—Needle, Plateau, Rugged, and Combined—each instantiated in both 1D and 2D. These landscapes span a diverse range of structural challenges for assessing algorithmic behavior. Here, we present two representative examples: a 1D needle landscape in Figure 1a and a 2D rugged landscape in Figure 1b.

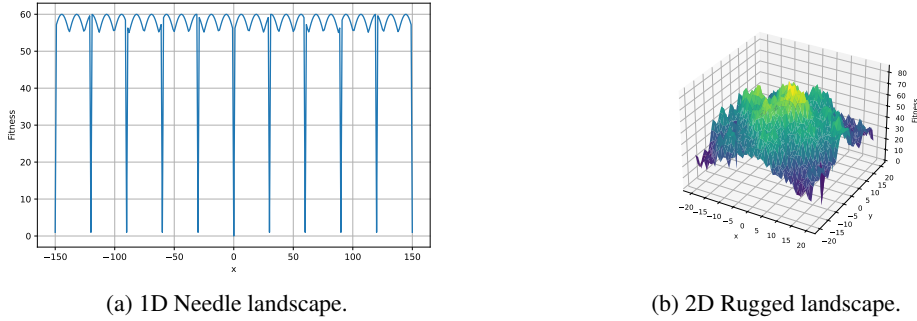


Figure 1: Examples of representative synthetic fitness landscapes used in the pilot study.

Additional landscapes are included in Appendix A for completeness.

Hyperparameter Settings All methods share a common time budget and initialization conditions. The full hyperparameter configuration for HC, HC-SHIFT, and GA is summarized in Appendix B.

4.2 Results

4.2.1 Success Rates Across Test Landscapes

While all algorithms succeed on the 1D landscapes, GA fails to converge within the time budget on the more structurally difficult 2D Needle, Rugged, and Combined landscapes.

Dim.	Landscape	GA	HC	HC-SHIFT
1D	Needle	p	p	p
	Plateau	p	p	p
	Rugged	p	p	p
	Combined	p	p	p
2D	Needle	f	p	p
	Plateau	p	p	p
	Rugged	f	p	p
	Combined	f	p	p

Table 1: Success (p) or failure (f) of GA, HC, and HC-SHIFT on synthetic landscapes. Shading indicates cases where GA fails within the time budget.

4.2.2 Analysis of Trial Counts (Table 2) and NFE (Table 3)

HC-SHIFT requires markedly fewer trials and fitness evaluations than standard HC across all synthetic landscapes, with the performance gap widening substantially in 2D settings. While HC exhibits restart-intensive behavior—incurring thousands of trials and tens of thousands of evaluations on rugged or needle-like 2D functions—SHIFT converges rapidly by accumulating compression information that progressively narrows the effective search space. These results demonstrate that SHIFT scales considerably more reliably than HC as dimensionality and landscape complexity increase.

Dim.	Landscape	HC	HC-SHIFT
1D	Needle	47	1
	Plateau	8	1
	Rugged	47	1
	Combined	47	1
2D	Needle	8198	1
	Plateau	8	1
	Rugged	11690	407
	Combined	1463	543

Table 2: Number of trials required for convergence on 1D and 2D synthetic landscapes.

Dim.	Landscape	HC	HC-SHIFT
1D	Needle	173	5
	Plateau	326	93
	Rugged	112	23
	Combined	164	28
2D	Needle	55119	12
	Plateau	372	140
	Rugged	38905	14160
	Combined	6883	5926

Table 3: Number of fitness evaluations (NFE) executed until convergence on 1D and 2D synthetic landscapes.

4.2.3 Case Studies on 2D Landscapes

To better illustrate the qualitative differences between HC and HC-SHIFT, we present two representative case studies on 2D landscapes: a structurally simple needle-like surface and a highly rugged multimodal surface.

Case 1: Simple 2D Landscape (Needle 2D). As shown in Figure 2, SHIFT collapses the large flat or needle-like domain into a navigable structure, enabling direct convergence to the global basin in a single trial. In contrast, HC exhibits no directional signal in this landscape and wanders chaotically until repeatedly restarting, ultimately requiring 8,198 trials to locate the optimum.

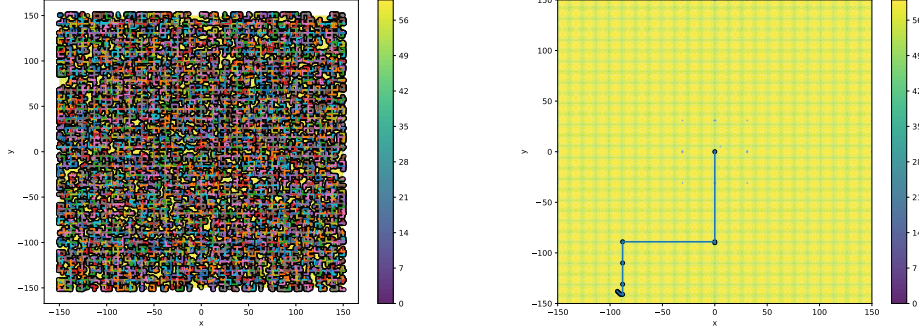


Figure 2: Search trajectories of HC (left) and HC-SHIFT (right) overlaid on the contour map of the Needle 2D landscape.

Case 2: Rugged High-Dimensional Landscape (Rugged 2D). On the highly irregular Rugged 2D landscape (Figure 3), HC becomes trapped almost immediately, requiring an extremely large number of restarts (11,690 trials) due to its inability to escape dense local minima. SHIFT, however, progressively compresses the landscape, reducing the effective search space at each restart and ultimately converging after 407 trials. The characteristic grid-like trajectory arises because SHIFT compresses each dimension independently, causing the algorithm to alternate axis-aligned moves as the underlying landscape is incrementally flattened.

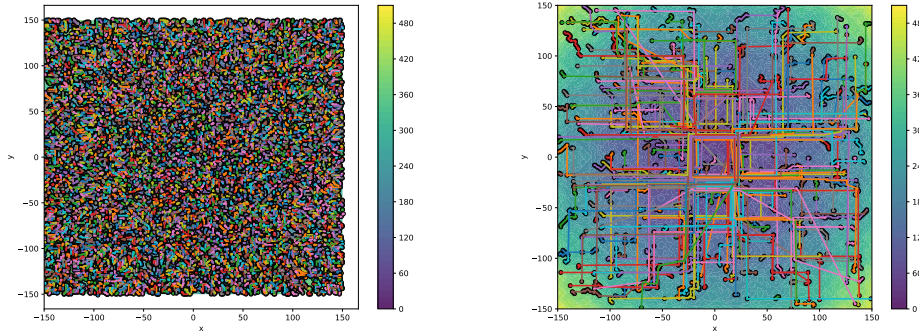


Figure 3: Search trajectories of HC (left) and HC-SHIFT (right) on the contour map of the Rugged 2D landscape.

Overall, the pilot study confirms that SHIFT’s compression-driven adaptation yields dramatically faster and more stable convergence than HC and GA, motivating its use in the following real-world experiments.

4.3 Benchmark Programs and Branch Extraction

Benchmark Programs To evaluate the efficacy of the proposed sigmoid-based compression, we constructed and curated a benchmark suite of 38 Python programs comprising 343 branching points. Semantically unreachable branches—such as the `False` direction of a range-based `for`-loop—are excluded from the count. These programs were deliberately designed or selected to exhibit complex control-flow dependencies and structurally challenging fitness landscapes, including rugged terrains, extended plateaus, needle-in-a-haystack configurations, and combinations thereof—landscape types that are well-known to be problematic for conventional SBST algorithms. The inherent difficulty of these benchmarks provides a rigorous testbed for assessing the performance gains afforded by our approach.

The benchmark programs are broadly classified into the following categories based on the primary challenge each presents to a search algorithm; the distribution is summarized in Table 4. The *Other* category encompasses programs with general control flow that do not fall into any of the specialized categories.

Category	# Programs	# Branch
Plateau	4	40
Rugged	3	22
Needle-in-a-haystack	3	13
Mixed and Complex	3	42
Other	25	226

Table 4: Benchmark categories and number of programs/branches.

Each category has its own special landscape feature, making it difficult to generate test cases automatically.

- **Plateau Landscapes**

Plateaus are regions in the search space where the fitness function yields a constant value, providing no gradient for a local search algorithm to follow.

- **Rugged Landscapes**

A rugged landscape is one where small changes to the input can cause large, unpredictable changes in the fitness value. This noise can mislead the search algorithm.

- **Needle-in-a-Haystack Landscapes**

These problems feature a fitness landscape that is almost entirely flat, with a single, extremely narrow basin of attraction leading to the solution. The algorithm receives little to no guidance until it is very close to the target.

- **Mixed and Complex Landscapes**

Several benchmarks combine multiple characteristics to create a composite challenge.

Branch Extraction For each benchmark program, we performed static analysis to identify all conditional branches connecting other program components. Each conditional statement (`if`, `while`, `for` and `match`) in the source code is identified as a branching point in the control-flow graph and stored to generate a test case for each branch. The goal of the test generation process is to generate inputs that cover these branches. The branch distance, a core component of the fitness function in SBST, is calculated at each conditional. For our experiment, we instrumented the Python code to report the outcome of each predicate, enabling the search algorithm to compute the fitness of a given input based on how closely it satisfied the conditions for a target branch. This instrumentation provides the necessary feedback for the search algorithm to navigate the program’s control flow.

4.4 Experimental Protocol

We evaluate three search algorithms—HC, HC-SHIFT, and GA—under a unified, time-budgeted per-branch evaluation framework. This ensures fairness across algorithms with different iteration costs and guarantees reproducibility through consistent initialization and random seed management.

4.4.1 Per-Branch Time Budget

Each conditional branch is tested independently under a strict wall-clock limit of $T = 20$ seconds. A fixed global random seed (42) is reapplied at the beginning of every branch evaluation. The timer starts immediately before the first call to the branch-level fitness function.

In HC and HC-SHIFT, multiple restart trials are executed sequentially until the branch-level time budget expires. Each trial begins from a newly sampled initial point and continues until a local optimum or an internal stopping condition is reached.

In GA, the population is initialized once and evolved continuously. Before evaluating any individual, a time check is performed; if the time budget is exceeded, GA stops immediately and returns the best individual observed so far. This strict time-guarding ensures a fair comparison between HC, HC-SHIFT, and GA.

4.4.2 Initialization Strategies

All algorithms share the same input initialization mechanism, applied per trial (HC, HC-SHIFT) or per individual (GA). Two initialization modes are supported:

Random Initialization. Each input variable is sampled uniformly from the automatically inferred domain $[L, H]$ derived from program constants.

Biased Initialization (Default). We extract variable-specific and global constants from the program’s AST and sample values using a mixture model:

$$x_0 \sim \begin{cases} \text{Uniform}(L, H), & \text{with prob. } 0.2, \\ \text{Gaussian}(c, \sigma^2), & \text{with prob. } 0.8, c \in \text{Constants}, \end{cases}$$

with σ set to 1% of the domain range.

4.4.3 Metrics

For every branch and target outcome, we record one CSV row per algorithm. HC and HC-SHIFT use the following fields:

```
function, lineno, outcome, convergence_speed, nfe,
best_fitness, best_solution, success, num_trials,
total_time, time_to_solution
```

GA uses the same fields plus:

```
generations
```

The fields have the following meaning:

- **function**: name of the function containing the target branch.
- **lineno**: branch identifier (bid) assigned during instrumentation (Section 3.2).
- **outcome**: target direction (True or False).
- **best_fitness**: minimum value of $(AL + nBD)$.
- **best_solution**: input that achieved the best fitness.
- **success**: True iff **best_fitness** is zero.
- **convergence_speed**: for HC, the number of accepted moves in X -space; for HC-SHIFT, the number of accepted moves in Z -space; for GA, the generation index where the best individual first appears.
- **nfe**: number of fitness evaluations.
- **num_trials**: for HC and HC-SHIFT, the number of restart trials attempted; for GA, the number of evaluated individuals.
- **generations** (GA only): number of completed generations.
- **total_time**: wall-clock time used (capped at T).
- **time_to_solution**: time until a successful input was found.

4.4.4 Hyperparameter Settings

Table 5 lists the hyperparameters used across all algorithms.

Component	Parameter	Value
Common	Time budget per branch T	20 s
	Random seed	42
	Input domain $[L, H]$	auto-detected (AST constants)
	Initialization mode	biased
HC	Max steps per trial	2000
	Move set	± 1 per dimension
HC-SHIFT	Max iterations per dimension	10
	Basin max search per dimension	1000
	Active-dimension updates	enabled
GA	Population size N	10000
	Tournament size k	3
	Elite ratio	0.1
	Mutation steps	$\{-3, -2, -1, 1, 2, 3\}$

Table 5: Hyperparameter settings for HC, HC-SHIFT, and GA.

Parameter Selection. These hyperparameters were determined through preliminary tuning experiments on representative benchmark subsets to ensure fair comparison:

- **GA population size:** Increased from 1,000 to 10,000 to provide sufficient diversity for exploring complex fitness landscapes within the 20-second time budget.
- **HC-SHIFT basin detection:** Tested configurations (10, 10000) and (100, 100) for (max iterations per dimension, basin max search per dimension). The selected (10, 1000) configuration achieved the best balance between exploration depth and computational efficiency.
- **HC parameters:** Retained standard settings (± 1 moves, 2000 max steps) as preliminary experiments confirmed adequate performance as a baseline method.

These settings serve as the default configuration across all experiments. Individual experiments (Section 4) may vary specific parameters to investigate particular research questions; such variations are explicitly noted in the corresponding experiment descriptions.

4.5 Experiment 1: Are Random Restarts Sufficient?

A natural question in search-based testing is whether simply increasing the number of random restart trials is sufficient to achieve high coverage within a fixed time budget. This experiment addresses that question by comparing the three algorithms introduced in Section 3—HC, GA, and HC-SHIFT—across all benchmark programs in the `./benchmark` directory, totaling 38 source files and 3,800 branch outcomes.

Each branch is tested independently under a strict per-branch time budget of 20 seconds (Section 4.4.1). Coverage is computed as

$$\text{Coverage} = \frac{\# \text{ branches covered}}{3800} \times 100\%.$$

Both initialization modes—biased and pure random (Section 4.4.2)—were evaluated for all algorithms.

Table 6 reports the aggregated results.

Algorithm	Init.	Coverage	Runtime
HC-SHIFT	biased	3642 / 3800 (95.84%)	3m 39s
HC-SHIFT	random	3569 / 3800 (93.92%)	4m 13s
HC	biased	3586 / 3800 (94.42%)	4m 32s
HC	random	3519 / 3800 (92.60%)	5m 03s
GA	biased	3519 / 3800 (92.60%)	4m 55s
GA	random	3491 / 3800 (91.87%)	4m 54s

Table 6: Coverage comparison under a 20-second per-branch time budget.

Key Findings. HC-SHIFT achieves the highest coverage across all configurations. With biased initialization it reaches **95.84%**, outperforming both HC and GA. Even under pure random initialization, it maintains strong performance (**93.92%**), demonstrating robustness to initialization quality.

Trial Efficiency: The Answer to Random Restarts. The critical distinction emerges when examining the number of trials required to achieve coverage. Table 7 presents selected challenging benchmarks (full results in Appendix E) where HC-SHIFT typically converges in **2–8 trials on average**, whereas HC and GA require **hundreds of thousands of trials** and still fail to achieve full coverage on plateau-dominated functions.

This efficiency gap demonstrates that *random restarts alone are insufficient*. Both HC and GA repeatedly sample similar regions without learning from prior failures, exhausting their time budget through redundant exploration.

Sub Benchmark	HC-SHIFT			HC			GA		
	Avg.#	Avg.t (s)	Cov.	Avg.#	Avg.t (s)	Cov.	Avg.#	Avg.t (s)	Cov.
count_divisor_2	2.58	4.145	100%	8312.83	6.674	88%	28765.00	6.829	88%
mixed_case	1.00	1.672	100%	13.67	5.782	96%	141325.50	8.451	78%
plateau2	3.75	0.572	100%	106216.25	5.005	88%	233190.88	5.056	88%
plateau_case	8.50	4.167	100%	86845.62	10.057	83%	267704.38	12.584	57%

Table 7: Selected challenging benchmarks showing HC-SHIFT’s superior trial efficiency (subset of full results in Appendix E).

Mechanisms Enabling HC-SHIFT’s Efficiency. HC-SHIFT’s superior performance arises from the interaction of several complementary mechanisms. First, the basin detection phase (Section 3.3.3(b)) explicitly identifies the geometric structure of flat or weakly varying regions, in contrast to HC and GA, which continue probing blindly. Once such basins are detected, the invertible SHIFT compression (Section 3.3.3(c)) contracts these plateaus into short intervals, enabling the algorithm to traverse areas that normally induce stagnation with only a few steps in the compressed space.

The restart policy further amplifies this effect: instead of restarting from arbitrary points, HC-SHIFT deliberately evaluates basin-boundary locations (Section 3.3.3(f), Algorithm 4), thereby exploring meaningful escape routes from local minima. Finally, compression metadata is preserved across trials (Section 3.3.3(d–e)), allowing information about previously explored regions to accumulate rather than being discarded. Together, these mechanisms convert naive random restarts into *informed restarts*, leading to higher coverage and dramatically reduced trial counts under the same time budget.

Impact of Initialization Strategies. Initialization strategy also affects performance, though to varying degrees across algorithms. Biased initialization (Section 4.4.2) benefits all methods, particularly GA, whose early generations depend heavily on the quality of initial samples. HC-SHIFT, however, shows the least sensitivity to initialization: even when starting from disadvantageous positions, the combination of basin detection and compression rapidly reshapes the landscape, steering the search toward productive areas. This robustness underscores that HC-SHIFT’s improvements stem primarily from its landscape transformation rather than from favorable initialization.

4.6 Performance on Complex Landscapes: Plateau and Rugged Cases

A central contribution of HC-SHIFT is its ability to reshape the fitness landscape in order to escape local optima and traverse plateaus efficiently. We evaluate this capability using parameterized benchmark functions designed to represent two archetypal landscape challenges common in software testing: stubborn local optima (Rugged) and large flat regions (Plateau).

4.6.1 Benchmark Specification

To rigorously evaluate the algorithms, we designed parameterizable benchmark functions that allow precise control over landscape difficulty—specifically, the length of a plateau or the frequency of local optima.

Plateau Benchmark: `schedule_cycle`. This function simulates a needle-in-a-haystack scenario common in scheduling logic, where a specific condition must be satisfied within a large, flat search space.

```
1 def schedule_cycle_N(timestamp: int) -> bool:
2     # A = cycle length (e.g., 7 days), B = granularity (e.g., 3600s)
3     if (timestamp // 86400) % A == (A - 1):
4         if (timestamp % 86400) // B == C:
5             return True
6     return False
```

By adjusting parameters A (cycle length in days) and B (granularity), we control the plateau length. A larger cycle or finer granularity produces a vast region with zero gradient information (fitness = 0 or constant), rendering standard gradient-based guidance ineffective.

Rugged Benchmark: `rugged_period`. This function introduces periodic noise to a simple linear distance function, creating multiple local optima (peaks and valleys) that trap gradient-based methods.

```
1 import math
2 TARGET = 31415
3
4 def rugged_period_N(x: int):
5     period = N # Controls ruggedness frequency
6     d = abs(x - TARGET)
7     # Add sinusoidal noise to create local optima
8     noise = int(10 * abs(math.sin((x - TARGET) / period)))
9     g = d + noise
10    if g == 0:
11        return 0 # Success
12    else:
13        return g # Fitness distance
```

The period parameter governs the density of local optima: a smaller value produces high-frequency noise with many peaks, while a larger value yields fewer, broader peaks.

4.6.2 Performance Analysis

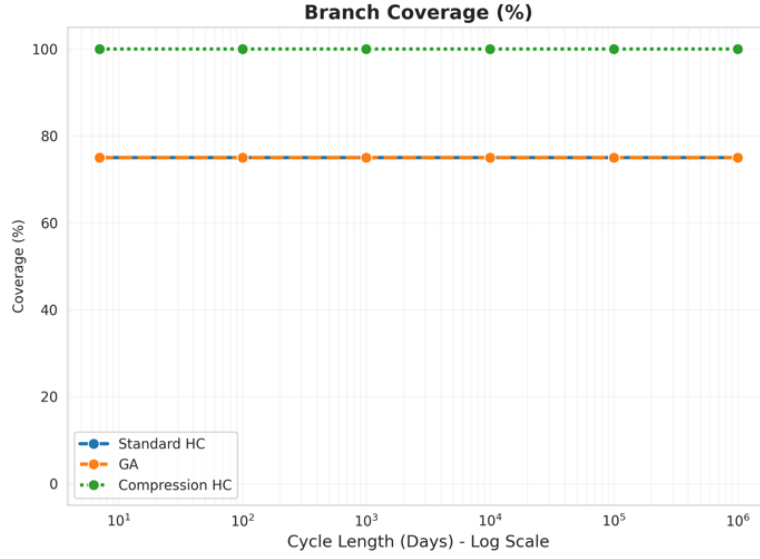


Figure 4: Performance comparison on plateau case

Plateau Analysis. As shown in Figure 4, standard HC fails on benchmarks with long plateau landscapes owing to the absence of gradient guidance. HC-SHIFT, by contrast, maintains full coverage regardless of plateau length. This robustness stems from its compression logic, which effectively “folds” flat regions—identified by equal fitness values—treating the entire plateau as a compact interval in the compressed space.

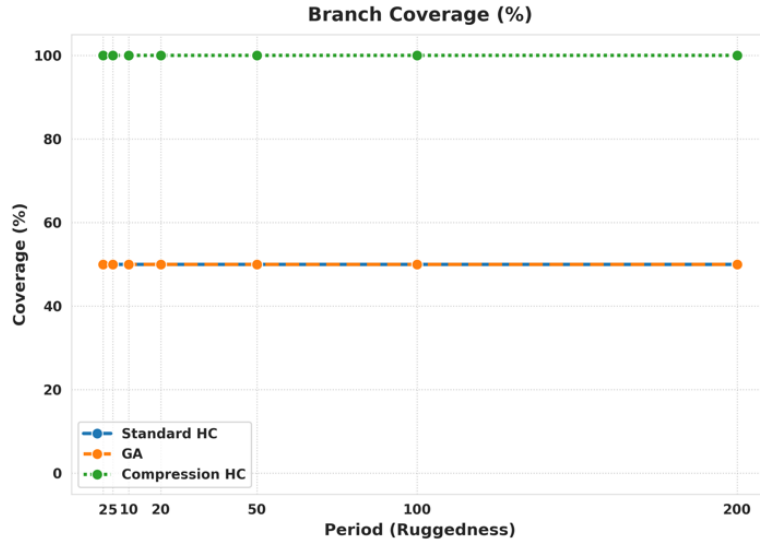


Figure 5: Performance comparison on rugged case

Rugged Analysis. Figure 5 illustrates the effect of landscape ruggedness, which is governed by the period parameter.

By varying this parameter, we control the number of local optima (peaks) within a fixed landscape extent, allowing us to measure each algorithm’s ability to escape local traps.

Standard HC and GA become trapped at the first local optimum they encounter. HC-SHIFT, by contrast, succeeds in these scenarios by identifying the basin of attraction of the local optimum, applying sigmoid compression to contract it, and stepping past the peak to resume descent toward the target.

The compression phase does incur additional computational cost, and failures were observed when the 20-second time budget proved insufficient. Nevertheless, given adequate time, HC-SHIFT achieves 100% coverage across all tested configurations, as shown in Figure 5.

4.7 Experiment 3: Necessity of ‘Active’ Dimension Strategy

While compression is effective, it incurs computational cost. To prevent time-budget exhaustion caused by compressing irrelevant dimensions—as observed in the high-frequency rugged case—we employ the `active_dim` strategy, which selectively restricts compression to dimensions that meaningfully influence fitness.

4.7.1 Benchmark Specification

To assess the robustness of HC-SHIFT under increasing input dimensionality, we selected a benchmark that all algorithms solve easily and augmented it with irrelevant input variables ranging from 1 to 200.

```
1 def active_easy_100(x0: int, x1: int, x2: int, ...,
2                     x98: int, x99: int):
3     # content of arbitrary1.py
4     return x
```

The growing number of irrelevant dimensions introduces spurious compression candidates, potentially exhausting the time budget on dimensions that do not contribute to fitness improvement.

4.7.2 Performance Analysis

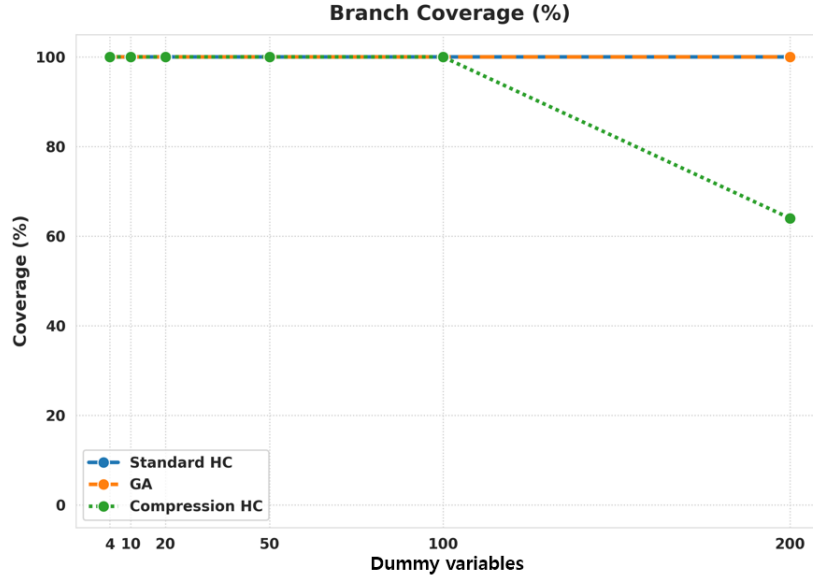


Figure 6: Effectiveness of Active Dimension logic against increasing irrelevant variables.

As shown in Figure 6, HC-SHIFT maintains 100% coverage on the vast majority of test configurations. The `active_dim` logic successfully filters out non-contributing dimensions, preventing unnecessary compression attempts and concentrating the time budget on the relevant subspace. A single failure is observed at 200 irrelevant variables, where the time budget is insufficient for the dimension-deactivation phase to complete. Nevertheless, HC-SHIFT remains substantially more robust than standard HC and GA, particularly on plateau and rugged landscapes.

4.8 Experiment 4: Seed Dependency Test

To evaluate the robustness of each method with respect to initialization randomness, we conduct a seed dependency test. Each method is executed with seeds ranging from 41 to 50, and for every run we measure the total execution time and final coverage. We report the mean, standard deviation, and coefficient of variation ($CV = \text{std}/\text{mean}$) to quantify each method’s sensitivity to seed variation.

Method	Time mean	Time std	Time CV	Cov mean	Cov std	Cov CV
SHIFT (biased)	228.50	20.97	0.092	95.78	0.18	0.002
HC (biased)	272.60	3.67	0.013	94.37	0.00	0.000
GA (biased)	292.20	5.23	0.018	92.67	0.13	0.001
SHIFT (random)	267.70	22.27	0.083	94.03	0.41	0.004
HC (random)	305.50	1.86	0.006	92.89	0.23	0.002
GA (random)	303.80	8.74	0.029	91.66	0.42	0.005

Table 8: Seed dependency results for biased and random initialization. CV denotes the coefficient of variation.

Results and Analysis. Across all configurations, SHIFT achieves the fastest average execution time and the highest coverage, demonstrating clear performance advantages over HC and GA. Although SHIFT exhibits greater variance than HC, its substantial gains in both coverage and convergence speed far outweigh this variability.

HC shows extremely low variance—a consequence of its deterministic behavior—but it consistently underperforms SHIFT in both efficiency and effectiveness. GA delivers the weakest performance, with lower coverage and slower execution than both HC and SHIFT, and its higher variance (especially under random initialization) indicates strong seed dependency.

These results indicate that SHIFT is the only method that simultaneously achieves high coverage, fast convergence, and robustness across seeds, whereas HC trades performance for stability and GA exhibits neither stability nor competitive performance.

5 Limitations and Future Works

Although the proposed sigmoid-based compression framework demonstrates clear benefits on rugged and plateau-heavy fitness landscapes, several limitations remain. Addressing these limitations will broaden the applicability of our method and facilitate deeper integration with a wider range of search and optimization problems.

Limited Input Domain and Program Support

The current implementation is restricted to Python programs that accept *integer* inputs. While this constraint simplified the design of the compression module, it considerably narrows the scope of programs amenable to analysis. Many real-world testing targets involve floating-point inputs, mixed numeric types, or structured data such as lists and records. Extending the compression scheme to operate reliably on continuous domains—in particular, floating-point inputs—is a direct and necessary next step.

Because floating-point domains introduce smoother yet potentially more deceptive search surfaces, the compression function may require additional refinements, such as adaptive scaling or stability constraints, to ensure consistent behavior. Future work could generalize the model to richer input types and more realistic program structures, thereby increasing the external validity of our approach.

Adaptive Maximum Compression Length

A key limitation of the current SHIFT framework is its reliance on a statically chosen maximum compression length. Because this cap is fixed beforehand, the model cannot fully exploit plateaus or wide basins that exceed the preset threshold, nor can it scale down compression when the terrain becomes narrow or irregular. This mismatch keeps SHIFT from fully adapting to the true geometry of each search region, leaving potential performance gains unused and forcing manual hyperparameter tuning.

A promising direction for addressing this issue is to make compression length adaptive through lightweight symbolic analysis performed before hill climbing. By examining guard conditions and extracting approximate constraint ranges for each variable, the model can predict where broad plateaus or complex basin structures are likely to appear. These symbolic estimates provide an informed prior on the landscape width, allowing the advanced SHIFT model to dynamically adjust its maximum compression length, expanding it for wide, flat regions and tightening it around constrained or nonlinear areas. This integration would make SHIFT genuinely landscape-aware and more robust across diverse programs without requiring hand-engineered compression parameters.

Applicability Beyond SBST: Hyperparameter Optimization

Our empirical analysis shows that compression significantly improves search in landscapes dominated by abrupt cliffs and large plateaus. These properties are also typical in hyperparameter optimization for machine learning or deep learning models, where many influential parameters, such as layer counts, batch sizes, or tree depths, are discrete and yield highly discontinuous fitness landscapes.

Given this similarity, our compression framework can naturally extend to *ML hyperparameter optimization*. Traditional hill climbing, greedy tuning, and random search methods often stall in local optima due to the lack of informative local gradients. By compressing the hyperparameter space and reducing the prevalence of unproductive local optima, our method may provide a lightweight yet effective enhancement to existing search loops. Integrating the compression transformation into general hyperparameter tuning pipelines represents a promising direction for future experimentation.

Overall, these limitations highlight meaningful paths forward. By expanding input support, applying the model to other optimization domains, or even integrating it with a wider class of metaheuristic algorithms, we aim to develop a general-purpose landscape compression framework that enhances search efficiency across diverse applications.

6 Conclusion

This work addresses a practical weakness of Search-Based Software Testing: traditional hill climbing stalls on rugged landscapes and extended plateaus because the fitness function provides little or no informative gradient. Our core motivation is straightforward—reshape the landscape rather than repeatedly failing on it. To that end, we introduce SHIFT, an invertible sigmoid-based compression framework that contracts flat or weakly varying regions, exposes actionable gradients, and enables even simple hill climbing to escape stagnation.

The core idea behind SHIFT is equally simple: detect basins, compress them, and restart outside. The algorithm first runs hill climbing in the original space, performs bidirectional basin detection to locate contiguous plateaus or near-flat regions, applies a smooth and fully invertible warping to shrink these basins in a separate coordinate space, and then continues the search using the compressed geometry. Along the way, SHIFT tracks active dimensions to avoid wasting time on irrelevant variables and accumulates compression metadata across restarts, effectively learning the landscape as it progresses.

The experiments validate the design across synthetic, constructed, and real benchmark programs. On synthetic landscapes, SHIFT consistently required fewer trials and evaluations than standard hill climbing, with the performance gap widening in higher dimensions. It handled needle-like, plateau-dominated, and rugged multimodal terrains with high reliability, while GA and HC both deteriorated or failed outright in the more extreme cases. On realistic SBST targets, SHIFT achieved the highest overall coverage under identical time budgets, showing strong gains on programs with structurally difficult branch conditions. Experiments on plateau scaling, ruggedness scaling, irrelevant-dimension injection, and seed dependency all reinforce the same conclusion: SHIFT remains effective and robust where conventional methods lose traction.

At the same time, several limitations remain. The current design fixes the maximum compression length statically and supports only integer-valued input domains. Moreover, the compression step can become expensive when many dimensions are present or when rugged regions require repeated detection cycles. Addressing these constraints—through adaptive compression length, symbolic pre-analysis, and broader input-type support—represents a clear path forward.

Overall, SHIFT demonstrates that landscape transformation is a lightweight yet powerful means of rehabilitating search in environments where gradient signals are easily misleading. By reshaping the geometry rather than redesigning the search operators, the method enables simple hill climbing to perform competitively on problems where it would ordinarily stall, offering a practical and extensible tool for SBST and, potentially, for broader classes of discrete optimization.

References

- Mark Harman and Phil McMinn. A theoretical and empirical analysis of evolutionary testing and hill climbing for structural test data generation. *IEEE Transactions on Software Engineering*, 36(2): 226–247, 2010. doi: 10.1109/TSE.2009.71. URL <https://philmcminn.com/publications/harman2007a.pdf>.
- Yongxiang Ma, Ermira Daka, Gordon Fraser, Michael Kuhn, and Pawel Liskowski. Scalable path search for automated test case generation. *Electronics*, 11(5):727, 2022. doi: 10.3390/electronics11050727. URL <https://www.mdpi.com/2079-9292/11/5/727>.
- Luca Manzoni, Luca Mariot, and Eva Tuba. Surfing on fitness landscapes: A boost on optimization by fourier surrogate modeling. *Applied Sciences*, 10(17):6089, 2020. doi: 10.3390/app10176089. URL <https://pmc.ncbi.nlm.nih.gov/articles/PMC7516743/>.

A Pilot Study(4.1): Full Synthetic Fitness Landscapes

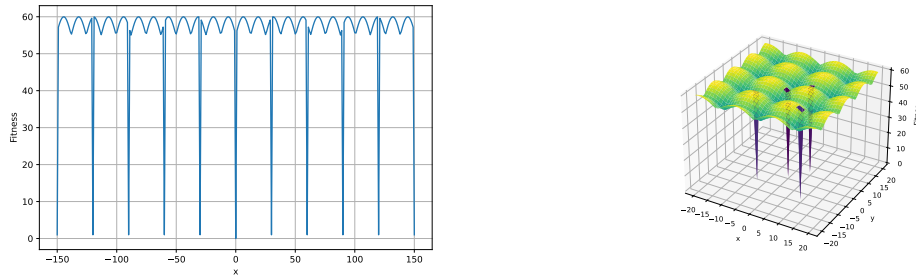


Figure 7: Needle landscapes in 1D and 2D.

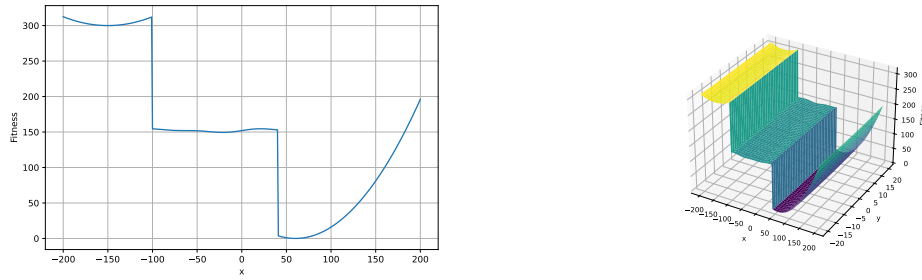


Figure 8: Plateau landscapes in 1D and 2D.

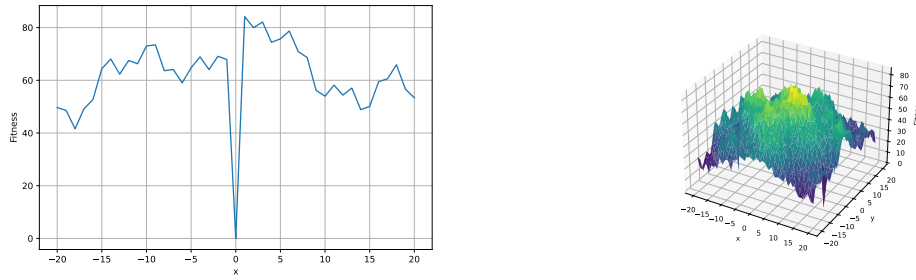


Figure 9: Rugged landscapes in 1D and 2D.

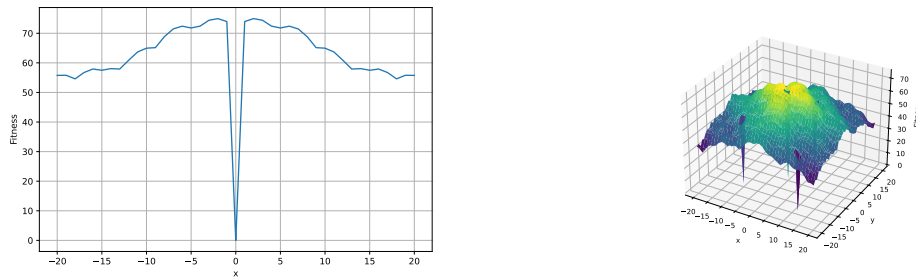


Figure 10: Combined landscapes in 1D and 2D.

B Pilot Study(4.1): Hyperparameter Settings

Component	Parameter	Value
Common	Time budget per branch T	20 s
	Random seed	42
	Input domain $[L, H]$	auto-detected (AST constants)
	Initialization mode	biased
HC	Max steps per trial	200
	Basin max search	100
	Move set	± 1 per dimension
HC-SHIFT	Max iterations per dimension	10
	Basin max search per dimension	100
	Active-dimension updates	enabled
GA	Population size N	1000
	Tournament size k	3
	Elite ratio	0.1
	Mutation steps	$\{-3, -2, -1, 1, 2, 3\}$

Table 9: Hyperparameter settings for HC, HC-SHIFT, and GA.

C Pseudocode for Baseline HC (Section 3.3.1)

Algorithm 1 Simple n -Dimensional Hill Climbing (HC)

Require: Fitness function F , start point x , dimension n , max steps K , time limit $t_{\max}$.

Ensure: Trajectory of visited points

```

1: Check time limit; if exceeded, return  $[(x, \infty)]$ 
2:  $f \leftarrow F(x)$ ,  $\text{traj} \leftarrow [(x, f)]$ 
3: for  $k = 1$  to  $K$  do
4:   Check time limit; if exceeded, return  $\text{traj}$ 
5:    $\mathcal{N} \leftarrow \emptyset$  ▷ axis-aligned neighbors
6:   for  $d = 1$  to  $n$  do
7:     Check time limit; if exceeded, return  $\text{traj}$ 
8:     Add  $(x_1, \dots, x_d - 1, \dots, x_n)$  to  $\mathcal{N}$ 
9:     Check time limit; if exceeded, return  $\text{traj}$ 
10:    Add  $(x_1, \dots, x_d + 1, \dots, x_n)$  to  $\mathcal{N}$ 
11:   end for
12:   Evaluate  $F(y)$  for all  $y \in \mathcal{N}$ 
13:    $(x', f') \leftarrow \arg \min_{y \in \mathcal{N}} F(y)$  ▷ steepest descent
14:   if  $f' < f$  then
15:      $x \leftarrow x'$ ,  $f \leftarrow f'$ 
16:     Append  $(x, f)$  to  $\text{traj}$ 
17:   else
18:     return  $\text{traj}$  ▷ local minimum reached
19:   end if
20: end for
21: return  $\text{traj}$ 

```

D Pseudocode for HC-SHIFT (Section 3.3.3)

Algorithm 2 HC-SHIFT: Main Algorithm

Require: Fitness function F , start point x , dimension n , max iterations $k_{\max}$, time limit $t_{\max}$, patience p .

- 1: $x \leftarrow$ initial point, $f \leftarrow F(x)$
- 2: $\mathcal{A} \leftarrow \{0, \dots, n-1\}$ ▷ active dimensions
- 3: $\sigma_d \leftarrow 0$ for all d ▷ stagnation counters
- 4: $\mathcal{CM} \leftarrow \text{CompressionManagerND}(n)$
- 5: **if** $f < \epsilon$ **then return** x ▷ early success
- 6: **end if**
- 7: **for** iteration $i = 1$ to $k_{\max}$ **do**
- 8: **if** $\mathcal{A} = \emptyset$ **or** time $> t_{\max}$ **then**
- 9: **return** best x
- 10: **end if**
- 11: **if** $f < \epsilon$ **then return** x
- 12: **end if**
- 13: $x, f \leftarrow \text{HILLCLIMBLOOP}(x, f, \mathcal{A}, \mathcal{CM}, \sigma, p)$ ▷ Alg. 5
- 14: **if** $f < \epsilon$ **then return** x
- 15: **end if**
- 16: $\mathcal{B} \leftarrow \text{DETECTBASINS}(x, \mathcal{A}, \mathcal{CM})$ ▷ Alg. 3
- 17: **if** $\mathcal{B} = \emptyset$ **then**
- 18: **return** best x ▷ no compressible basin
- 19: **end if**
- 20: $x, f \leftarrow \text{SELECTRESTARTPOINT}(x, \mathcal{B})$ ▷ Alg. 4
- 21: **if** $f < \epsilon$ **then return** x
- 22: **end if**
- 23: **end for**
- 24: **return** best x

Algorithm 3 DETECTBASINS: Basin Detection and Compression Update

Require: Current point x , active dimensions $\mathcal{A}$, compression manager $\mathcal{CM}$.

Ensure: Basin set $\mathcal{B}$ (each element: $(d, (b_{\text{start}}, b_{\text{len}}))$)

- 1: $\mathcal{B} \leftarrow \emptyset$
- 2: **for** each dimension $d \in \mathcal{A}$ **do**
- 3: Check time limit; **if** exceeded, **return** $\mathcal{B}$
- 4: $B_d \leftarrow \text{DETECT1DBASIN}(x, d)$ ▷ bidirectional search
- 5: **if** $B_d \neq \emptyset$ **then**
- 6: Get fixed coordinates: $\text{fixed} \leftarrow (x_i : i \neq d)$
- 7: $\mathcal{CM}.\text{UPDATEDIMENSION}(d, \text{fixed}, B_d)$
- 8: Add (d, B_d) to $\mathcal{B}$
- 9: **end if**
- 10: **end for**
- 11: **return** $\mathcal{B}$

Algorithm 4 SELECTRESTARTPOINT: Basin Boundary Evaluation

Require: Current point x , basin set $\mathcal{B}$.

Ensure: Restart point x_{restart} , fitness f_{restart}

```
1:  $\mathcal{R} \leftarrow \emptyset$  ▷ restart candidates
2: for each  $(d, (b_{\text{start}}, b_{\text{len}})) \in \mathcal{B}$  do
3:   Check time limit; if exceeded, return  $(x, F(x))$ 
4:    $b_{\text{end}} \leftarrow b_{\text{start}} + b_{\text{len}} - 1$ 
5:   Create point  $y^-$  by setting  $x_d \leftarrow b_{\text{start}} - 1$ 
6:   Add  $(y^-, F(y^-))$  to  $\mathcal{R}$ 
7:   Create point  $y^+$  by setting  $x_d \leftarrow b_{\text{end}} + 1$ 
8:   Add  $(y^+, F(y^+))$  to  $\mathcal{R}$ 
9: end for
10:  $(x_{\text{restart}}, f_{\text{restart}}) \leftarrow \arg \min_{(y, f_y) \in \mathcal{R}} f_y$ 
11: return  $(x_{\text{restart}}, f_{\text{restart}})$ 
```

Algorithm 5 HILLCLIMBLOOP: Compressed-Space Hill Climbing

Require: Current point x , fitness f , active dims $\mathcal{A}$, compression manager $\mathcal{CM}$, stagnation σ , patience p .

Ensure: Updated point x , fitness f

```
1: step_count  $\leftarrow 0$ 
2: repeat
3:   Check time limit; if exceeded, return  $(x, f)$ 
4:    $\mathcal{N} \leftarrow \text{GENERATENEBHORS}(x, \mathcal{A}, \mathcal{CM})$  ▷ Alg. 6
5:    $x_{\text{best}}, f_{\text{best}} \leftarrow x, f$ 
6:    $\mathcal{M} \leftarrow \emptyset$  ▷ meaningful dimensions
7:   for each  $(y, f_y, \mathcal{D}_y) \in \mathcal{N}$  do
8:     if  $f_y < f_{\text{best}}$  then
9:        $x_{\text{best}}, f_{\text{best}} \leftarrow y, f_y$ 
10:    end if
11:    if  $f_y \neq f$  then
12:       $\mathcal{M} \leftarrow \mathcal{M} \cup \mathcal{D}_y$ 
13:    end if
14:  end for
15:  Update stagnation:  $\sigma_d \leftarrow \begin{cases} 0 & \text{if } d \in \mathcal{M} \\ \sigma_d + 1 & \text{otherwise} \end{cases}$ 
16:  Deactivate:  $\mathcal{A} \leftarrow \mathcal{A} \setminus \{d : \sigma_d \geq p\}$ 
17:  if  $f_{\text{best}} < f$  then
18:     $x \leftarrow x_{\text{best}}, f \leftarrow f_{\text{best}}$ 
19:    step_count  $\leftarrow \text{step\_count} + 1$ 
20:  else
21:    break ▷ local minimum
22:  end if
23: until step_count  $\geq \text{max\_steps}$ 
24: return  $(x, f)$ 
```

Algorithm 6 GENERATENEIGHBORS: Compressed Neighbor Generation

Require: Current point x , active dimensions $\mathcal{A}$, compression manager $\mathcal{CM}$.

Ensure: Neighbor set $\mathcal{N}$ (each element: $(y, f_y, \mathcal{D}_y)$)

```
1:  $\mathcal{N} \leftarrow \emptyset$ 
2: for each  $d \in \mathcal{A}$  do ▷ Axis-aligned neighbors
3:   Get compression system  $w_d$  for dimension  $d$  (if exists)
4:   if  $w_d$  exists then
5:      $z_d \leftarrow w_d(x_d)$ 
6:      $n^- \leftarrow w_d^{-1}(z_d - 1), n^+ \leftarrow w_d^{-1}(z_d + 1)$ 
7:   else
8:      $n^- \leftarrow x_d - 1, n^+ \leftarrow x_d + 1$ 
9:   end if
10:  Create neighbors  $y^-$  and  $y^+$  by modifying  $x$  at dimension  $d$ 
11:  Add  $(y^-, F(y^-), \{d\})$  and  $(y^+, F(y^+), \{d\})$  to  $\mathcal{N}$ 
12: end for
13: if  $|\mathcal{A}| \geq 2$  then ▷ Diagonal neighbors
14:   for each pair  $(d_1, d_2)$  in combinations of  $\mathcal{A}$  do
15:     Get compression systems  $w_{d_1}, w_{d_2}$  (if exist)
16:     Compute neighbor values for  $d_1$ :  $\{n_1^-, n_1^+\}$  (compressed or not)
17:     Compute neighbor values for  $d_2$ :  $\{n_2^-, n_2^+\}$  (compressed or not)
18:     for  $v_1 \in \{n_1^-, n_1^+\}, v_2 \in \{n_2^-, n_2^+\}$  do
19:       Create diagonal neighbor  $y$  by modifying  $x$  at dims  $d_1, d_2$ 
20:       Add  $(y, F(y), \{d_1, d_2\})$  to  $\mathcal{N}$ 
21:     end for
22:   end for
23: end if
24: return  $\mathcal{N}$ 
```

E Complete Benchmark Results (4.5)

Benchmark	HC-SHIFT		HC		GA	
	Avg. Trials	Avg. Time (s)	Avg. Trials	Avg. Time (s)	Avg. Trials	Avg. Time (s)
arbitrary1	1.00	0.008	1.00	0.006	18.42	0.188
arbitrary2	1.00	0.448	1.29	0.004	5.86	0.149
arbitrary3	1.00	0.009	1.00	0.008	20.00	0.141
arbitrary4	1.00	0.006	1.38	0.006	9.00	0.162
arbitrary5	1.00	0.007	1.00	0.007	6.50	0.119
arbitrary6	1.17	1.243	1.33	0.004	1.67	0.106
arbitrary7	1.00	0.634	1.67	0.007	4.00	0.163
arbitrary8	17.67	3.340	4583.50	3.337	4488.67	3.496
arbitrary9	1.00	0.011	1.11	0.005	2.00	0.143
arbitrary10	1.00	0.013	1.12	0.006	207.75	0.147
collatz_step	1.00	0.015	6.00	0.008	9.67	0.086
combined1	1.00	0.356	2.21	0.493	4417.86	0.476
combined2	8.19	16.738	38292.06	12.536	247545.00	12.599
count_divisor_1	3.08	5.108	9350.92	5.004	6038.17	5.103
count_divisor_2	2.58	4.145	8312.83	6.674	28765.00	6.829
derivative_quadratic	70.00	10.008	264995.00	10.002	7139.50	10.051
digit_sum	1.00	0.006	1.50	0.006	9.80	0.082
ex1	1.00	0.008	1.00	0.007	23.62	0.190
ex2	1.00	0.019	1.00	0.019	2.33	0.133
ex3	1.00	0.018	1.07	0.010	31.07	0.319
ex4	1.00	0.047	1.00	0.024	5202.50	0.264
ex5	2.33	2.161	5.00	0.227	69590.17	2.733
ex6	1.00	0.009	2.17	0.010	64.83	0.041
ex7	1.00	0.003	1.00	0.004	5.00	0.037
mixed_case	1.00	1.672	13.67	5.782	141325.50	8.451
needle1	1.00	0.293	1.80	0.824	2.60	0.279
needle2	51.50	10.007	16628.75	10.002	259999.50	10.102
needle_case	29.75	10.012	131752.50	10.002	176919.75	10.043
parallel_test	1.12	0.288	4.31	0.010	29.58	0.057
plateau1	1.00	0.101	1.00	0.037	28.38	0.163
plateau2	3.75	0.572	106216.25	5.005	233190.88	5.056
plateau3	1.00	0.053	1.00	0.021	129301.81	6.435
plateau_case	8.50	4.167	86845.62	10.057	267704.38	12.584
prime_check	1.00	0.024	2.75	0.005	5.58	0.052
rugged1	1.00	0.028	1.00	0.017	2.06	0.261
rugged2	53.25	10.010	17306.00	10.003	295836.75	10.063
rugged_case	1.00	1.143	242500.00	10.002	16167.50	10.071
triangle	1.83	2.524	8.08	0.012	20.00	0.220

Table 10: Per-benchmark average number of trials and total time (in seconds) for biased initialization, aggregated over all branches in each file.